

Python Exception Handling

1. Exception क्या होता है?

जब Python program run हो रहा होता है और अचानक कोई ऐसी problem आती है जिसके कारण program सामान्य तरीके से आगे नहीं चल पाता, उसे **Exception** कहते हैं।

उदाहरण:

```
age = int(input("Enter your age: "))
```

अगर user लिखता है:

Enter your age: hello

तो Python error देगा:

ValueError

क्योंकि "hello" को integer में convert नहीं किया जा सकता।

आसान भाषा में:

Exception = Program चलाते समय आने वाली **problem**

2. Exception Handling क्या है?

Program में आने वाली exceptions को पहचानकर उन्हें properly handle करने की process को **Exception Handling** कहते हैं।

इसका मुख्य उद्देश्य है:

Program को **error** आने पर **crash** होने से बचाना।

उदाहरण:

अगर game में user गलत input देता है, तो game बंद नहीं होना चाहिए। उसे message देना चाहिए और दोबारा खेलने का मौका देना चाहिए।

3. try और except

Exception handling के लिए सबसे basic keywords हैं:

try
except

Basic structure:

```
try:  
    # जिस code में error आ सकता है  
    risky_code()
```

```
except:  
    # error आने पर यह code चलेगा  
    handle_error()
```

याद रखने की **trick**:

try → कोशिश करो
except → अगर problem आए तो संभालो

4. Simple Example

```
try:  
    number = int(input("Enter a number: "))  
    print("Your number:", number)
```

```
except:  
    print("Please enter a number!")
```

अगर user लिखता है:

Enter a number: 10

Output:

Your number: 10

लेकिन अगर user लिखता है:

Enter a number: hello

Output:

Please enter a number!

Program crash नहीं होगा।

5. Game Example – Guess the Number

```
import random
```

```
number = random.randint(1, 10)
```

```
while True:
```

```
    try:
```

```
        guess = int(input("Guess 1-10: "))
```

```
        if guess == number:
```

```
            print("You Win!")
```

```
            break
```

```
        else:
```

```
            print("Wrong Guess!")
```

```
    except:
```

```
        print("Only numbers allowed!")
```

यह **Game** कैसे काम करता है?

Step 1

```
import random
```

`random` module को import किया गया है।

Step 2

```
number = random.randint(1, 10)
```

Computer 1 से 10 के बीच एक random number choose करता है।

उदाहरण:

Computer number = 7

Step 3

while True:

Game को बार-बार चलाने के लिए infinite loop बनाया गया है।

Step 4

try:

हम Python को कहते हैं:

इस code को चलाने की कोशिश करो।

Step 5

```
guess = int(input("Guess 1-10: "))
```

User से number लिया जाता है।

`input()` से मिलने वाली value string होती है, इसलिए `int()` का उपयोग करके उसे integer में बदला जाता है।

Step 6

```
if guess == number:
```

Check किया जाता है कि user का guess सही है या नहीं।

अगर सही है:

You Win!

और:

```
break
```

game को समाप्त कर देता है।

Step 7

अगर guess गलत है:

```
print("Wrong Guess!")
```

Game दोबारा guess करने का मौका देता है।

Step 8

अगर user number की जगह text डालता है:

Guess 1-10: hello

तो `int("hello")` error देगा।

लेकिन `except` उसे handle कर लेगा:

```
except:
```

```
    print("Only numbers allowed!")
```

Output:

Only numbers allowed!

और game बंद नहीं होगा।

6. ValueError

जब किसी value को गलत format में convert करने की कोशिश की जाती है, तो अक्सर `ValueError` आता है।

Example:

```
int("hello")
```

यह `ValueError` देगा।

हम इसे specifically handle कर सकते हैं:

```
try:
    number = int(input("Enter number: "))

except ValueError:
    print("Numbers only!")
```

Game में:

```
try:
    guess = int(input("Guess: "))

except ValueError:
    print("Please enter a valid number!")
```

7. Specific Exception Handle करना

सिर्फ `except` लिखने के बजाय हम particular exception का नाम भी दे सकते हैं।

Example:

```
try:
    number = int(input("Enter number: "))

except ValueError:
    print("Invalid number!")
```

यह बेहतर तरीका है क्योंकि हमें पता है कि कौन-सी exception handle की जा रही है।

Important:

`except ValueError:`

का मतलब:

अगर `ValueError` आए, तो यह code चलाओ।

8. ZeroDivisionError

अगर किसी number को zero से divide किया जाए:

10 / 0

तो Python में:

ZeroDivisionError

आता है।

Example:

try:

```
a = int(input("Number 1: "))
```

```
b = int(input("Number 2: "))
```

```
print(a / b)
```

except ZeroDivisionError:

```
print("Zero se divide nahi kar sakte!")
```

अगर user:

Number 1: 10

Number 2: 0

देता है, तो output होगा:

Zero se divide nahi kar sakte!

Program crash नहीं होगा।

9. Multiple except

एक program में अलग-अलग types की exceptions हो सकती हैं।

Example:

try:

```
a = int(input("Number 1: "))
```

```
b = int(input("Number 2: "))
```

```
print(a / b)
```

```
except ValueError:
```

```
    print("Numbers only!")
```

```
except ZeroDivisionError:
```

```
    print("Zero se divide nahi kar sakte!")
```

अगर **user** लिखता है:

Number 1: hello

तो:

Numbers only!

अगर **user** लिखता है:

Number 1: 10

Number 2: 0

तो:

Zero se divide nahi kar sakte!

10. else

else तब चलता है जब **try** block में कोई error नहीं आता।

Example:

```
try:
```

```
    number = int(input("Enter number: "))
```

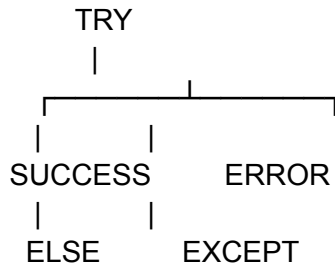
```
except ValueError:
```

```
    print("Wrong input!")
```

```
else:
```

```
    print("Correct input!")
```

Flow:



यानी:

Error नहीं आया → else

Error आया → except

11. finally

`finally` block हमेशा execute होता है।

चाहे error आए या न आए।

Example:

```
try:  
    number = int(input("Enter number: "))
```

```
except ValueError:  
    print("Wrong input!")
```

```
finally:  
    print("Game Finished!")
```

अगर input सही है:

```
Enter number: 10  
Game Finished!
```

अगर input गलत है:

```
Enter number: hello  
Wrong input!  
Game Finished!
```

दोनों cases में:

Game Finished!

print होगा।

12. Complete Exception Handling Structure

Python में complete structure:

try:

Code

except:

Error आने पर

else:

Error नहीं आने पर

finally:

हमेशा चलेगा

लेकिन हर program में **else** और **finally** लगाना जरूरी नहीं है।

13. Real-Life Example

Exception Handling को real life से समझें।

Imagine करें आप school में game खेल रहे हैं।

TRY

↓

Game खेलने की कोशिश

↓

क्या problem आई?

↓

YES → EXCEPT → Problem handle करो

↓

NO → आगे game खेलो

इसलिए:

try = कोशिश करना

except = problem संभालना

else = अगर problem नहीं आई

finally = हर हालत में करना

14. Important Exceptions

Exception	कब आती है?
<code>ValueError</code>	गलत value/type conversion
<code>ZeroDivisionErr</code> or <code>or</code>	Zero से divide करने पर
<code>TypeError</code>	गलत data types के साथ operation
<code>IndexError</code>	List में गलत index
<code>KeyError</code>	Dictionary में मौजूद न होने वाली key
<code>NameError</code>	Undefined variable इस्तेमाल करने पर

15. Student Challenge – Guessing Game

अब students को यह challenge दें:

एक **Guess the Number game** बनाओ जिसमें:

1. Computer 1–10 के बीच random number चुने।
2. User number guess करे।
3. सही guess पर `You Win!` दिखे।
4. गलत guess पर `Try Again!` दिखे।
5. अगर user text enter करे तो program crash न हो।
6. `ValueError` को specifically handle करो।

7. User को दोबारा number enter करने का मौका मिले।

Expected output:

🎯 GUESS THE NUMBER

Guess 1-10: hello
Numbers only!

Guess 1-10: 4
Try Again!

Guess 1-10: 8
You Win!

16. Quick Revision

Exception क्या है?

Program run करते समय आने वाली error/problem.

Exception Handling क्या है?

Errors को properly handle करके program को crash होने से बचाना।

try?

जिस code में error आ सकता है उसे try करने के लिए।

except?

Error आने पर उसे handle करने के लिए।

else?

जब try सफल हो जाए।

finally?

हर situation में execute होता है।

सबसे **basic syntax**:

try:

code

except:

handle_error()

Golden Rule

Good programs सिर्फ सही **input** के लिए नहीं बनते। **Good programs** गलत **input** को भी **safely handle** करते हैं।